BVRIT HYDERABAD

College of Engineering for Women
Department of CSE (AI & ML)



CASE STUDY

Hotel Management System

Database Management System

TEAM: 5

25WH1A6669 – Kurra Sridevi

25WH1A6676 – Bhooreddygari Praharsha

25WH1A6683 – Unguturi Praghnya

25WH1A6690 – Gowragalla Vaishnavi

25WH1A6697 – Mutyala Jashwitha

25WH1A66A4 – Badike Saathvika

25WH1A66B2 – Rentala Sruthi

25WH1A66B9 – Boinapally Sirija

25WH1A66C6 – Chebiyyam Sri Yuktha

26WH5A6612 – Radapaka Deepika

Under the guidance of

Miss V. Asha (Assistant Professor)

INDEX

S.NO.	TOPIC	PAGE NO.
1	Introduction	3
2	Requirement Analysis	8
3	ER Diagram	18
4	Database Implementation	22
5	SQL Queries and Outputs	28

1. INTRODUCTION

1.1 Overview of the Project

The Hotel Management System is a database-based project developed to manage the day-to-day activities of a hotel in an organized and efficient way. A hotel deals with several tasks, such as maintaining guest details, managing room bookings, checking room availability, handling payments, and keeping employee records. Managing all this information manually can be difficult, especially when the number of guests and reservations increases.

This project aims to make these tasks easier by storing the required information in a centralized database. Instead of maintaining separate records for every activity, the system organizes information into different tables that are connected to one another.

The Hotel Management System uses Database Management System (DBMS) concepts to store, update, and retrieve information whenever required. It helps hotel staff keep track of reservations, room details, guest information, and payments while reducing the chances of errors and duplicate records.

The main focus of this project is to design a simple and well-structured database that supports the basic operations of a hotel.

1.2 Purpose of the Project

The main purpose of the Hotel Management System is to simplify the management of hotel records and make information easier to access and maintain.

The objectives of this project are:

- To store and manage guest information in an organized manner.
- To maintain details of hotel rooms, their types, and availability.
- To record room bookings along with check-in and check-out dates.
- To keep track of payments made by guests.
- To maintain employee details and their roles in the hotel.
- To maintain hotel inventory records and identify supplies that need restocking.
- To reduce manual work and avoid unnecessary duplication of data.
- To retrieve and update hotel information easily using SQL queries.
- To maintain accurate records by establishing relationships between different tables.

1.3 Scope of the Project

The Hotel Management System covers the basic operations involved in managing a hotel. It includes guest registration, room management, reservations, payment records, employee information, and additional hotel services.

The system allows hotel staff to add new guest details, check room availability, create bookings, update reservation information, and record payments. It also helps maintain information about different room categories and the services offered by the hotel.

The project mainly focuses on database design and implementation using SQL. Operations such as inserting new records, updating existing details, deleting unnecessary records, and retrieving information can be performed through SQL queries.

The database can also be used to find details such as available rooms, booking history, payment information, and total revenue collected by the hotel.

1.4 Tables and Attributes

The Hotel Management System consists of six main tables. Each table stores information related to a particular part of hotel management.

1. Guests Table

The Guest table stores the personal and contact details of customers who visit or make reservations at the hotel.

Attribute	Description
guest_id (PK)	Unique ID assigned to each guest
full_name	Full name of the guest
phone	Contact number of the guest
email	Email address of the guest
city	City where the guest resides

2. Rooms Table

The Room table stores details of individual rooms in the hotel. It helps maintain room numbers, categories, and their current status.

Attribute	Description
room_id (PK)	Unique ID assigned to each room

room_number	Room number
room_type	Type of room, such as Single, Double, Deluxe, or Suite
price_per_night	Price charged per night
room_status	Current availability status of the room

3. Staff Table

The Staff table maintains information about hotel employees, including their roles, contact details, and salaries.

Attribute	Description
staff_id (PK)	Unique ID assigned to each staff member
staff_name	Name of the staff members
job_role	Role or designation of the employee
phone	Contact number
salary	Salary of the staff member

4. Reservations Table

The Reservations table stores information about room bookings made by guests. It maintains check-in and check-out dates along with the booking status.

Attribute	Description
reservation_id (PK)	Unique ID assigned to each reservation
guest_id (FK)	Refers to the Guests table
room_id (FK)	Refers to the Rooms table

staff_id (FK)	Refers to the Staff table
check_in	Scheduled check-in date
check_out	Scheduled check-out date
booking_status	Current status of the reservation

5. Bills Table

The Bills table stores billing and payment details related to hotel reservations.

Attribute	Description
bill_id (PK)	Unique ID assigned to each bill
reservation_id (FK)	Refers to the Reservations table
amount	Total bill amount
payment_method	Method of payment, such as Cash, Card, or UPI
payment_status	Current payment status
bill_date	Date on which the bill was recorded

6. Inventory Table

The Inventory table stores details of supplies used in hotel operations, such as towels, bedsheets, toiletries, and cleaning materials. It helps maintain stock quantities and identify items that need restocking.

Attribute	Description
item_id (PK)	Unique ID assigned to each inventory item
item_name	Name of the inventory item

quantity	Available quantity of the item
unit	Unit of measurement, such as pieces or bottles
reorder_level	Minimum stock level at which restocking is required

1.5 Relationships Between Tables

The tables in the Hotel Management System are connected using Primary Keys and Foreign Keys. These relationships help maintain accurate records and reduce unnecessary duplication of data.

- **Guests and Reservations:** One guest can make multiple reservations, but each reservation belongs to one guest.
- **Rooms and Reservations:** One room can be reserved multiple times on different dates, but each reservation is linked to one room.
- **Staff and Reservations:** One staff member can handle multiple reservations, but each reservation is associated with one staff member.
- **Reservations and Bills:** One reservation can have multiple billing records, but each bill belongs to one reservation.

The Inventory table stores hotel supply details independently and does not have a direct foreign key relationship with the other tables.

1.6 Conclusion

The Hotel Management System is designed to make hotel data easier to manage by storing information in a structured database. It brings together important details such as guest records, room information, bookings, payments, employees, and services.

Through this project, DBMS concepts such as tables, Primary Keys, Foreign Keys, relationships, and SQL queries are applied to a real-world example. The system provides a simple and organized way to maintain hotel records and perform common database operations.

2. REQUIREMENT ANALYSIS

Requirement Analysis is an important part of developing the Hotel Management System, as it helps identify the features and conditions needed for its proper functioning. It focuses on understanding how hotel activities such as guest registration, room management, reservations, billing, and inventory maintenance can be handled through a structured database. The requirements are divided into functional, non-functional, and integration requirements to ensure that the system operates efficiently, maintains accurate records, and supports smooth coordination between different hotel operations.

The Hotel Management System uses MySQL to manage guest records, room details, staff information, reservations, billing transactions, and hotel inventory. The database is designed to organize these records, maintain relationships between them, and support efficient data retrieval through SQL operations.

2.1 Functional Requirements

Functional requirements define the operations and activities that the Hotel Management System must support. The system uses MySQL to store, retrieve, update, and manage hotel information through SQL commands.

2.1.1 Guest Management

- The system shall allow the insertion, retrieval, updating, and deletion of guest records.
- Each guest shall be identified by a unique Guest_ID.
- The system shall maintain guest information, including full name, phone number, email address, and city.
- The system shall support searching for guests using their identification numbers, names, or contact details.
- The system shall allow retrieval of guest information along with their reservation details using SQL JOIN operations.

2.1.2 Room Management

- The system shall maintain information about hotel rooms, including room number, room type, price per night, and room status.
- Each room shall be identified by a unique Room_ID.
- Room numbers shall be maintained as unique values to avoid duplicate room records.
- The system shall support different room categories, such as Single, Double, Deluxe, and Suite.
- The system shall maintain room availability statuses, including Available, Occupied, and Maintenance.

- The system shall allow updating room status whenever necessary.
- The system shall support retrieving rooms based on availability, room type, and price range.

2.1.3 Reservation Management

- The system shall allow the creation, retrieval, updating, and cancellation of reservation records.
- Each reservation shall be identified by a unique Reservation_ID.
- Every reservation shall be associated with a valid guest and room.
- The system shall maintain reservation information, including check-in date, check-out date, and booking status.
- Each reservation shall contain a reference to the staff member responsible for handling it.
- The system shall support different booking statuses, such as Booked, Checked-In, Checked-Out, and Cancelled.
- The system shall allow retrieval of reservation details along with guest names and room numbers.
- The system shall support calculating the duration of a reservation using SQL date functions.

2.1.4 Check-in and Check-out Management

- The system shall maintain the scheduled check-in and check-out dates of guests.
- The system shall support identifying reservations that have reached the check-in or check-out stage.
- The booking status shall indicate whether a reservation is Booked, Checked-In, Checked-Out, or Cancelled.
- The system shall allow updating booking status as the guest's stay progresses.
- Room availability information shall be maintained separately through the Rooms table.
- The system shall support retrieving guest and room information associated with completed and active reservations.

2.1.5 Billing and Payment Management

- The system shall maintain billing information for hotel reservations.
- Each bill shall be identified by a unique Bill_ID.
- Every bill shall be associated with a valid reservation through a foreign key.
- The system shall store billing details, including amount, payment method, payment status, and bill date.
- The system shall support payment methods such as Cash, Card, and UPI.
- The system shall maintain payment statuses such as Paid and Pending.
- The system shall allow updating payment status when a payment is completed.
- The system shall support retrieving billing information along with guest and reservation details.
- The system shall calculate total paid revenue using SQL aggregate functions.
- The system shall support identifying pending bills and bills exceeding a specified amount.

2.1.6 Staff Management

- The system shall maintain information about hotel employees.
- Each staff member shall be identified by a unique Staff_ID.
- The system shall store staff details, including name, job role, phone number, and salary.
- The system shall support different employee roles, such as Manager, Receptionist, Housekeeping, and Accountant.
- The system shall associate reservation records with the staff members responsible for them.
- The system shall support retrieving staff information based on job roles and salary conditions.
- The system shall allow updating employee details whenever required.

2.1.7 Inventory Management

- The system shall maintain information about hotel supplies and inventory items.
- Each inventory item shall be identified by a unique Item_ID.
- The system shall store item details, including item name, quantity, unit, and reorder level.
- The system shall support inventory items such as toiletries, bedsheets, towels, cleaning materials, and other hotel supplies.
- The system shall allow inserting, updating, retrieving, and deleting inventory records.
- The system shall identify inventory items whose available quantities are less than or equal to their reorder levels.
- The system shall support calculating total inventory quantities based on units of measurement.
- The system shall help hotel staff identify items that require restocking.

2.1.8 Room Availability and Status Management

- The system shall maintain the current operational status of every room.
- The Rooms table shall store room availability information.
- The system shall support retrieving rooms marked as Available.
- Rooms marked as Maintenance shall be identifiable as temporarily unavailable.
- The system shall allow authorized database operations to update room statuses.
- Reservation information shall be maintained separately in the Reservations table.
- The system should verify room availability for the requested reservation period before confirming new bookings.

2.1.9 Data Retrieval and Report Generation

- The system shall support retrieving information from database tables using SELECT statements.
- The system shall allow filtering records using WHERE conditions.
- The system shall support sorting records using ORDER BY.
- The system shall retrieve related information from multiple tables using SQL JOIN operations.
- The system shall support calculating totals and averages using aggregate functions such as COUNT, SUM, AVG, MIN, and MAX.
- The system shall support grouping reservation and inventory information using GROUP BY.
- The system shall generate query-based results related to room availability, reservations, billing, staff, and inventory.

- The system shall support identifying the most expensive room using subqueries and aggregate functions.
- The system shall calculate reservation durations using the DATEDIFF function.

2.1.10 Data Modification Operations

- The system shall support inserting new records into the database using INSERT statements.
- The system shall allow modifying existing information using UPDATE statements.
- The system shall support deleting records using DELETE statements when appropriate.
- The system shall allow updating room availability and payment statuses.
- The system shall support adding new guest records without affecting existing guest information.
- Data modification operations shall follow the constraints defined for the corresponding tables.

2.1.11 Search and Filtering Operations

- The system shall support searching for guest details based on specified conditions.
- The system shall allow identifying rooms that are currently available.
- The system shall support filtering rooms based on their price per night.
- The system shall allow retrieving reservations associated with particular guests.
- The system shall support identifying bills with Pending payment status.
- The system shall allow filtering staff records based on salary.
- The system shall identify inventory items requiring restocking.
- The system shall support combining search conditions with sorting operations to display organized results.

2.1.12 Data Validation and Integrity

- The system shall uniquely identify records using primary keys.
- The system shall maintain relationships between related tables using foreign keys.
- The system shall prevent duplicate values in attributes defined with UNIQUE constraints.
- The system shall enforce mandatory data entry through NOT NULL constraints.
- The system shall apply CHECK constraints to attributes such as room prices, staff salaries, and inventory quantities where defined.
- The system shall ensure that reservations reference existing guests, rooms, and staff members.
- The system shall ensure that bills reference valid reservation records.
- The database shall reject SQL operations that violate implemented integrity constraints.

2.2 Non-Functional Requirements

Non-functional requirements specify the quality standards and operational conditions that the Hotel Management System should satisfy.

2.2.1 Performance

- The database should retrieve guest, room, reservation, billing, staff, and inventory information within a reasonable response time.
- SQL queries should be efficient and avoid unnecessary processing.
- Common operations such as checking available rooms and retrieving reservation details should execute without significant delays.
- Performance should remain acceptable as the number of guests, reservations, and billing records increases.

2.2.2 Reliability

- The database should store and retrieve hotel information accurately and consistently.
- Completed SQL operations should be recorded correctly without unexpected data loss.
- Reservation and billing information should remain consistent during database operations.
- Failed operations should not leave incomplete changes when appropriate transaction controls are used.
- The system should minimize errors caused by invalid or inconsistent records.

2.2.3 Security

- Guest, staff, reservation, and billing information should be protected against unauthorized access.
- MySQL user accounts and privileges should control database access where configured.
- Sensitive information, including guest contact details and employee salaries, should be protected from unauthorized modification.
- Database operations should be performed only by users with appropriate permissions.
- Access to important records should follow suitable database security practices.

2.2.4 Data Integrity and Consistency

- Primary keys and foreign keys should maintain valid relationships between database tables.
- Constraints such as NOT NULL, UNIQUE, and CHECK should be applied wherever required.
- The database should prevent invalid references between reservations, guests, rooms, staff, and bills.
- Room prices, billing amounts, staff salaries, and inventory quantities should follow the defined validation constraints.
- Database records should remain consistent after insertion, updating, and deletion operations.

2.2.5 Usability

- Database operations should be understandable and manageable through the MySQL command-line interface.
- Tables and attributes should follow meaningful and consistent naming conventions.
- SQL statements should be organized and easy to understand.
- Query results should display information in a clear and readable format.

- SQL error messages should help users identify invalid operations or constraint violations.

2.2.6 Scalability

- The database should support increasing numbers of guests, rooms, reservations, employees, and billing transactions.
- Additional hotel records should be accommodated without major changes to the database structure.
- Efficient SQL queries should support future growth in the volume of stored information.
- The relational database design should allow additional hotel management features to be introduced when required.

2.2.7 Maintainability

- The database should follow a clear and organized relational structure.
- Table definitions, constraints, and SQL statements should be properly documented.
- The system should support modifications to existing records without unnecessarily affecting unrelated tables.
- Future improvements should be possible with minimal changes to the existing database design.
- Consistent table and attribute names should make the database easier to understand and maintain.

2.2.8 Backup and Recovery

- Database backups should be created when required to protect important hotel records.
- Backup files should support restoration in case of accidental deletion, system failure, or data corruption.
- Suitable MySQL backup and recovery utilities should be used when necessary.
- Recovery procedures should help restore guest, reservation, billing, staff, and inventory information.
- Regular backups should reduce the risk of permanent data loss.

2.2.9 Availability

- Hotel information should be accessible whenever the MySQL server is operational.
- Authorized users should be able to perform database operations during the intended working period.
- The database should support access to important records whenever required for hotel operations.
- Unnecessary interruptions should be minimized to maintain continuity of database activities.

2.2.10 Compatibility

- The database should function correctly with the selected MySQL version and operating system.

- SQL commands, data types, and constraints should be compatible with MySQL.
- Database operations should be supported through the MySQL command-line environment.
- The database structure should support future integration with suitable hotel management applications when required.

2.2.11 Transaction Management

- Related database operations should be grouped into transactions when consistency is required.
- COMMIT should save completed database changes.
- ROLLBACK should cancel uncommitted changes when necessary.
- Transaction handling should help prevent partial updates to related reservation and billing records.
- Transactions should support reliable execution of database modifications.

2.2.12 Auditability

- Reservation records should identify the staff members responsible for handling bookings.
- Billing records should maintain payment amounts, methods, statuses, and dates.
- Stored reservation and billing information should support the review of hotel transactions.
- Inventory records should support the verification of available quantities and reorder levels.
- Detailed tracking of individual database modifications would require additional audit-logging functionality.

2.3 Integration Requirements

Integration requirements define how different components of the Hotel Management System exchange and maintain related information through the MySQL database.

2.3.1 Key and Referential Integrity Requirements

- Each table should have an appropriate primary key to uniquely identify its records.
- Foreign keys should connect related tables and prevent invalid references.
- The Reservations table should maintain references to the Guests, Rooms, and Staff tables.
- The Bills table should maintain a reference to the Reservations table.
- Related records should maintain referential integrity during insertion, updating, and deletion operations.

2.3.2 Guest and Reservation Integration

- Each reservation should be associated with an existing guest record.
- Guest information should be maintained separately from reservation details.
- The Guest_ID foreign key in the Reservations table should reference the corresponding record in the Guests table.

- SQL JOIN operations should support retrieving reservation details along with guest names and contact information.
- The database should support identifying guests who have made reservations.

2.3.3 Room and Reservation Integration

- Each reservation should be associated with a valid room record.
- Room details should be maintained in the Rooms table.
- The Room_ID foreign key in the Reservations table should reference the corresponding room.
- SQL JOIN operations should support retrieving room numbers along with reservation details.
- Room availability should be checked against relevant reservation records before confirming a booking.
- Room status changes should be performed using appropriate SQL UPDATE statements.

2.3.4 Staff and Reservation Integration

- Each reservation should maintain a reference to the staff member responsible for handling it.
- Staff information should be stored separately in the Staff table.
- The Staff_ID foreign key in the Reservations table should reference an existing staff record.
- The database should support retrieving reservations handled by a particular staff member.
- SQL JOIN operations should allow staff information to be combined with reservation details when required.

2.3.5 Reservation and Billing Integration

- Every bill should be associated with a valid reservation.
- The Reservation_ID foreign key in the Bills table should reference the Reservations table.
- Billing information should be maintained separately from reservation details.
- SQL JOIN operations should support retrieving reservation information along with bill amounts and payment statuses.
- The database should support identifying pending bills associated with guest reservations.
- Payment status updates should be performed through appropriate SQL statements.

2.3.6 Room Status and Booking Integration

- The Rooms table should maintain the current operational status of each room.
- The Reservations table should maintain booking details, including check-in and check-out dates.
- Room availability queries should support identifying rooms marked as Available.
- Reservation dates should be considered when determining whether a room can be booked for a requested period.
- The system should maintain consistency between room status and reservation information through appropriate database operations.
- Automatic synchronization of room statuses would require additional database logic or application functionality.

2.3.7 Inventory and Hotel Operations Integration

- The Inventory table should maintain details of supplies required for hotel operations.
- Each inventory record should contain item name, quantity, unit, and reorder level.
- SQL queries should support identifying items that require restocking.
- Inventory quantities should be updated through appropriate SQL statements when supplies are received or used.
- Inventory information should support the hotel's housekeeping and general supply management activities.
- The current database maintains inventory records independently without direct foreign key relationships to the reservation or billing tables.

2.3.8 Data Retrieval and Reporting Integration

- Information from related tables should be combined using SQL JOIN operations.
- Reservation reports should retrieve information from the Reservations, Guests, and Rooms tables.
- Billing reports should combine information from the Bills and Reservations tables.
- Aggregate functions should support calculating total revenue, average room prices, and reservation counts.
- GROUP BY operations should support summarizing reservation statuses and inventory quantities.
- SQL queries should support filtering and sorting records according to reporting requirements.

2.3.9 Data Consistency and Error Handling

- Database constraints should prevent invalid references and unsupported data values.
- MySQL should report errors when SQL operations violate implemented constraints.
- Transactions should be used where necessary to maintain consistency across related database updates.
- Reservation and billing modifications should preserve the relationships between their corresponding records.
- Room status and inventory quantity changes should be performed using explicit SQL operations unless automatic update logic has been implemented.
- The system should avoid deleting records that are referenced by other tables unless the defined foreign key rules permit the operation.

3. ER DIAGRAM

The Entity Relationship (ER) Diagram represents the structure of the Hotel Management System database. It shows how guest details, room information, staff records, reservations, billing details, and inventory are organized into different entities. Primary keys and foreign keys are used to establish relationships between the tables and maintain data consistency.

3.1 Entity Relationship Diagram

The ER diagram consists of six entities: Guests, Rooms, Staff, Reservations, Bills, and Inventory. The Reservations table connects guests, rooms, and staff members, while the Bills table stores billing information related to reservations. The Inventory table maintains hotel supplies independently.

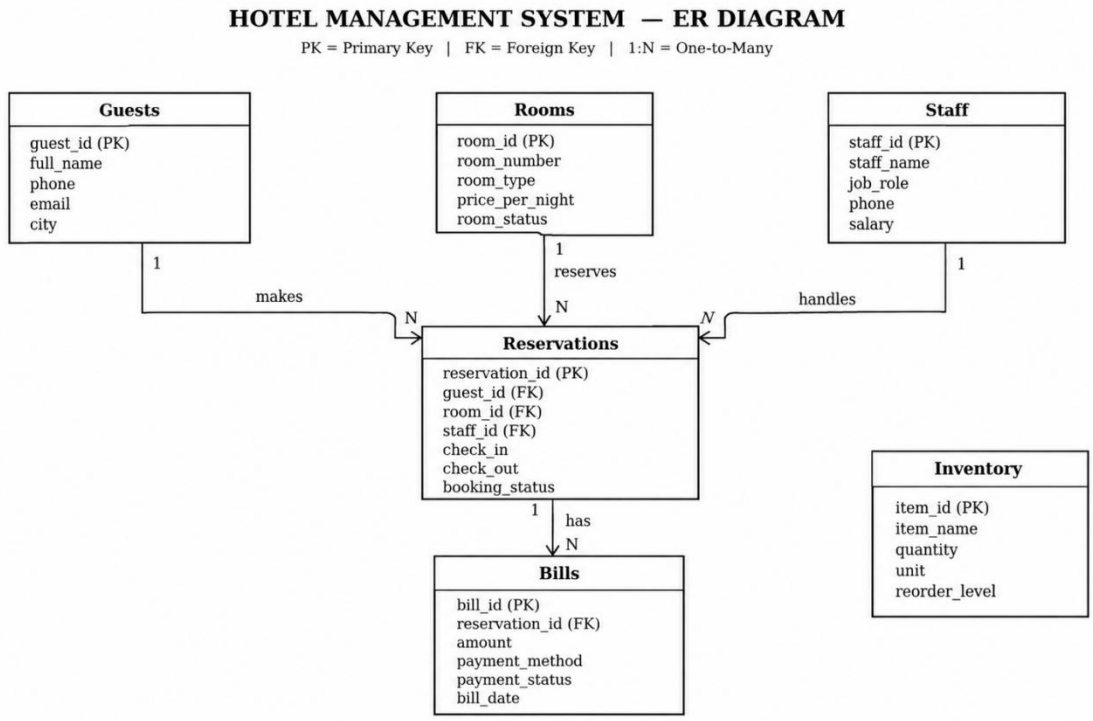


Figure 3.1: ER diagram of the Hotel Management System.

3.2 Entities and Key Identification

The database contains eight entities. Each entity stores information about a specific part of hotel operations. PK uniquely identifies a record, while FK references a related record in another entity.

Key notation: PK – Primary Key; FK – Foreign Key; CK – Candidate Key; Composite Key – a key made from two or more attributes.

Entity	Primary Key (PK)	Important Attributes	Foreign Key(s)
Guest	GuestID	FullName, Phone, Email, Address	—
Room	RoomID	RoomNumber, Floor, Status	RoomTypeID
Reservations	reservation_id	check_in, check_out, booking_status	guest_id, room_id, staff_id
Staff	StaffID	FullName, Role, Phone, Email	—
Bills	bill_id	amount, payment_method, payment_status, bill_date	bill_datereservation_id
Inventory	item_id	item_name, quantity, unit, reorder_level	—

Entity Descriptions

1. Guests

The Guests entity stores the personal and contact information of customers staying at the hotel. Each guest is identified by a unique guest_id.

2. Rooms

The Rooms entity maintains information about hotel rooms, including room numbers, room types, prices per night, and availability statuses. Each room has a unique room_id.

3. Staff

The Staff entity stores employee information, including names, job roles, contact numbers, and salaries. Each staff member is identified by a unique staff_id.

4. Reservations

The Reservations entity stores booking details such as check-in date, check-out date, and booking status. It is connected to the Guests, Rooms, and Staff entities through foreign keys.

5. Bills

The Bills entity maintains billing information, including bill amounts, payment methods, payment statuses, and bill dates. Each bill is linked to a reservation through reservation_id.

6. Inventory

The Inventory entity stores information about hotel supplies, including item names, quantities, units, and reorder levels. It helps maintain stock records and identify supplies that require restocking.

3.3 Relationships and Cardinalities

Cardinality defines the number of records in one entity that can be associated with records in another entity. The Hotel Management System contains four direct one-to-many relationships.

No.	Related Entities	Connecting Key	Cardinality
1	Guests – Reservations	Reservations.guest_id	1
2	Rooms – Reservations	Reservations.room_id	1
3	Staff – Reservations	Reservations.staff_id	1
4	Reservations – Bills	Bills.reservation_id	1

Explanation of Major Relationships

1. Guests – Reservations

One guest can make multiple reservations, but each reservation belongs to one guest. The guest_id foreign key in the Reservations table connects each booking to the corresponding guest.

2. Rooms – Reservations

One room can be reserved multiple times on different dates, but each reservation refers to one room. The room_id foreign key in the Reservations table establishes this relationship.

3. Staff – Reservations

One staff member can handle multiple reservations, while each reservation is associated with one staff member. The staff_id foreign key in the Reservations table identifies the employee responsible for the booking.

4. Reservations – Bills

One reservation can have multiple billing records, but each bill is linked to one reservation. The reservation_id foreign key in the Bills table establishes this relationship.

Indirect Many-to-Many Relationship

Guests – Rooms

A guest can reserve different rooms, and a room can be reserved by different guests over time. This relationship is maintained through the Reservations table, which stores the guest ID, room ID, and booking details.

Independent Entity

Inventory

The Inventory entity maintains hotel supply records separately. It does not have a direct foreign key relationship with the other entities in the current database design.

3.4 Important Design Notes

- Every entity has a primary key that uniquely identifies its records.
- Foreign keys are used to connect related tables and maintain referential integrity.
- The Reservations table connects Guests, Rooms, and Staff.
- The Bills table is linked to Reservations through reservation_id.
- The Inventory table stores hotel supply details independently.
- Room availability should be checked against reservation dates to avoid overlapping bookings.
- The database structure supports the storage, retrieval, and updating of hotel information using SQL commands.

Conclusion

The ER diagram provides a clear representation of the Hotel Management System database. It shows how guest information, room details, staff records, reservations, and billing information are connected, while inventory records are maintained separately. The use of primary keys, foreign keys, and defined relationships helps organize hotel data and maintain consistency across the database.

4. DATABASE IMPLEMENTATION

Database implementation is the process of creating a database and its tables using SQL commands. In the Hotel Management System, MySQL is used to store and manage guest details, room information, staff details, reservations, billing information, and inventory records.

4.1 Database Creation

The database `HotelManagementDB` is created using the `CREATE DATABASE` command. The `USE` command selects the database for further operations.

SQL Command:

CREATE DATABASE HotelManagementDB;
USE HotelManagementDB;

```
mysql> CREATE DATABASE HotelManagementDB;
Query OK, 1 row affected (0.03 sec)

mysql> USE HotelManagementDB;
Database changed
```

Figure 4.1: Creation and selection of the Hotel Management database.

4.2 Table Creation

Six tables are created in the Hotel Management database to store the information required for hotel operations. Primary keys are used to uniquely identify records, while foreign keys establish relationships between related tables.

4.2.1 Guests Table

The Guests table stores guest information, including guest ID, full name, phone number, email address, and city.

```
mysql> CREATE TABLE Guests (
    ->     guest_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     full_name VARCHAR(100) NOT NULL,
    ->     phone VARCHAR(15) UNIQUE,
    ->     email VARCHAR(100) UNIQUE,
    ->     city VARCHAR(60) NOT NULL
    -> );
Query OK, 0 rows affected (0.10 sec)
```

```
mysql> INSERT INTO Guests (full_name, phone, email, city) VALUES
    -> ('Aarav Sharma','9876500001','aarav@example.com','Hyderabad'),
    -> ('Priya Reddy','9876500002','priya@example.com','Vijayawada'),
    -> ('Rahul Kumar','9876500003','rahul@example.com','Warangal'),
    -> ('Sneha Rao','9876500004','sneha@example.com','Hyderabad'),
    -> ('Arjun Patel','9876500005','arjun@example.com','Chennai');
Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Guests;
+----------+--------------+------------+-------------------+------------+
| guest_id | full_name    | phone      | email             | city       |
+----------+--------------+------------+-------------------+------------+
|        1 | Aarav Sharma | 9876500001 | aarav@example.com | Hyderabad  |
|        2 | Priya Reddy  | 9876500002 | priya@example.com | Vijayawada |
|        3 | Rahul Kumar  | 9876500003 | rahul@example.com | Warangal   |
|        4 | Sneha Rao    | 9876500004 | sneha@example.com | Hyderabad  |
|        5 | Arjun Patel  | 9876500005 | arjun@example.com | Chennai    |
+----------+--------------+------------+-------------------+------------+
5 rows in set (0.00 sec)
```

Figure 4.2: Guests table and its records.

4.2.2 Rooms Table

The Rooms table stores room details such as room number, room type, price per night, and room availability status.

```
mysql> CREATE TABLE Rooms (
    ->     room_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     room_number VARCHAR(10) NOT NULL UNIQUE,
    ->     room_type ENUM('Single','Double','Deluxe','Suite') NOT NULL,
    ->     price_per_night DECIMAL(10,2) NOT NULL CHECK (price_per_night > 0),
    ->     room_status ENUM('Available','Occupied','Maintenance')
    ->         NOT NULL DEFAULT 'Available'
    -> );
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> INSERT INTO Rooms (room_number, room_type, price_per_night, room_status) VALUES
    -> ('101','Single',1500,'Available'),
    -> ('102','Double',2500,'Occupied'),
    -> ('201','Deluxe',4000,'Occupied'),
    -> ('202','Suite',6500,'Available'),
    -> ('301','Double',2800,'Maintenance'),
    -> ('302','Deluxe',4200,'Available');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Rooms;
+---------+-------------+-----------+-----------------+-------------+
| room_id | room_number | room_type | price_per_night | room_status |
+---------+-------------+-----------+-----------------+-------------+
|       1 | 101         | Single    |         1500.00 | Available   |
|       2 | 102         | Double    |         2500.00 | Occupied    |
|       3 | 201         | Deluxe    |         4000.00 | Occupied    |
|       4 | 202         | Suite     |         6500.00 | Available   |
|       5 | 301         | Double    |         2800.00 | Maintenance |
|       6 | 302         | Deluxe    |         4200.00 | Available   |
+---------+-------------+-----------+-----------------+-------------+
6 rows in set (0.00 sec)
```

Figure 4.3: Rooms table and its records.

4.2.3 Staff Table

The Staff table contains staff details, including staff ID, name, job role, phone number, and salary.

```
mysql> CREATE TABLE Staff (
    ->     staff_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     staff_name VARCHAR(100) NOT NULL,
    ->     job_role VARCHAR(50) NOT NULL,
    ->     phone VARCHAR(15) UNIQUE,
    ->     salary DECIMAL(10,2) NOT NULL CHECK (salary >= 0)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> INSERT INTO Staff (staff_name, job_role, phone, salary) VALUES
    -> ('Kiran Rao','Manager','9000000001',45000),
    -> ('Meena Das','Receptionist','9000000002',25000),
    -> ('Vikram Singh','Housekeeping','9000000003',20000),
    -> ('Anjali Verma','Accountant','9000000004',30000);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Staff;
+----------+--------------+--------------+------------+----------+
| staff_id | staff_name   | job_role     | phone      | salary   |
+----------+--------------+--------------+------------+----------+
|        1 | Kiran Rao    | Manager      | 9000000001 | 45000.00 |
|        2 | Meena Das    | Receptionist | 9000000002 | 25000.00 |
|        3 | Vikram Singh | Housekeeping | 9000000003 | 20000.00 |
|        4 | Anjali Verma | Accountant   | 9000000004 | 30000.00 |
+----------+--------------+--------------+------------+----------+
4 rows in set (0.00 sec)
```

Figure 4.4: Staff table and its records.

4.2.4 Reservations Table

The Reservations table stores booking details, including guest ID, room ID, staff ID, check-in date, check-out date, and booking status. Foreign keys connect reservations with the Guests, Rooms, and Staff tables.

```
mysql> CREATE TABLE Reservations (
    ->     reservation_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     guest_id INT NOT NULL,
    ->     room_id INT NOT NULL,
    ->     staff_id INT,
    ->     check_in DATE NOT NULL,
    ->     check_out DATE NOT NULL,
    ->     booking_status ENUM('Booked','Checked-In','Checked-Out','Cancelled')
    ->         NOT NULL DEFAULT 'Booked',
    ->     FOREIGN KEY (guest_id) REFERENCES Guests(guest_id),
    ->     FOREIGN KEY (room_id) REFERENCES Rooms(room_id),
    ->     FOREIGN KEY (staff_id) REFERENCES Staff(staff_id),
    ->     CHECK (check_out > check_in)
    -> );
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> INSERT INTO Reservations
    -> (guest_id, room_id, staff_id, check_in, check_out, booking_status) VALUES
    -> (1,2,2,'2026-10-10','2026-10-12','Checked-In'),
    -> (2,3,1,'2026-10-11','2026-10-14','Booked'),
    -> (3,1,2,'2026-10-01','2026-10-03','Checked-Out'),
    -> (4,4,1,'2026-10-15','2026-10-17','Booked'),
    -> (5,6,2,'2026-09-20','2026-09-22','Checked-Out');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Reservations;
+----------------+----------+---------+----------+------------+------------+----------------+
| reservation_id | guest_id | room_id | staff_id | check_in   | check_out  | booking_status |
+----------------+----------+---------+----------+------------+------------+----------------+
|              1 |        1 |       2 |        2 | 2026-10-10 | 2026-10-12 | Checked-In     |
|              2 |        2 |       3 |        1 | 2026-10-11 | 2026-10-14 | Booked         |
|              3 |        3 |       1 |        2 | 2026-10-01 | 2026-10-03 | Checked-Out    |
|              4 |        4 |       4 |        1 | 2026-10-15 | 2026-10-17 | Booked         |
|              5 |        5 |       6 |        2 | 2026-09-20 | 2026-09-22 | Checked-Out    |
+----------------+----------+---------+----------+------------+------------+----------------+
5 rows in set (0.00 sec)
```

Figure 4.5: Reservations table and its records.

4.2.5 Bills Table

The Bills table maintains billing information for reservations, including bill amount, payment method, payment status, and bill date.

```
mysql> CREATE TABLE Bills (
    ->     bill_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     reservation_id INT NOT NULL UNIQUE,
    ->     amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0),
    ->     payment_method ENUM('Cash','Card','UPI','Online') NOT NULL,
    ->     payment_status ENUM('Paid','Pending','Refunded')
    ->         NOT NULL DEFAULT 'Pending',
    ->     bill_date DATE NOT NULL,
    ->     FOREIGN KEY (reservation_id) REFERENCES Reservations(reservation_id)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> INSERT INTO Bills
    -> (reservation_id, amount, payment_method, payment_status, bill_date) VALUES
    -> (1,5000,'UPI','Paid','2026-10-10'),
    -> (2,12000,'Card','Pending','2026-10-11'),
    -> (3,3000,'Cash','Paid','2026-10-03'),
    -> (4,13000,'Online','Pending','2026-10-15'),
    -> (5,8400,'UPI','Paid','2026-09-22');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Bills;
+---------+---------------+----------+---------------+----------------+------------+
| bill_id | reservation_id | amount   | payment_method | payment_status | bill_date  |
+---------+---------------+----------+---------------+----------------+------------+
|       1 |              1 |  5000.00 | UPI            | Paid           | 2026-10-10 |
|       2 |              2 | 12000.00 | Card           | Pending        | 2026-10-11 |
|       3 |              3 |  3000.00 | Cash           | Paid           | 2026-10-03 |
|       4 |              4 | 13000.00 | Online         | Pending        | 2026-10-15 |
|       5 |              5 |  8400.00 | UPI            | Paid           | 2026-09-22 |
+---------+---------------+----------+---------------+----------------+------------+
5 rows in set (0.00 sec)

mysql> SELECT * FROM Inventory;
```

Figure 4.6: Bills table and its records.

4.2.6 Inventory Table

The Inventory table stores information about hotel supplies, including item name, quantity, unit, and reorder level.

```
mysql> CREATE TABLE Inventory (
    ->     item_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     item_name VARCHAR(100) NOT NULL UNIQUE,
    ->     quantity INT NOT NULL CHECK (quantity >= 0),
    ->     unit VARCHAR(20) NOT NULL,
    ->     reorder_level INT NOT NULL DEFAULT 5 CHECK (reorder_level >= 0)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> INSERT INTO Inventory (item_name, quantity, unit, reorder_level) VALUES
    -> ('Bath Towels',25,'pieces',10),
    -> ('Soap Bars',8,'pieces',10),
    -> ('Shampoo Bottles',15,'bottles',5),
    -> ('Bed Sheets',6,'pieces',8),
    -> ('Water Bottles',50,'bottles',20);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Inventory;
+---------+-----------------+----------+---------+---------------+
| item_id | item_name       | quantity | unit    | reorder_level |
+---------+-----------------+----------+---------+---------------+
|       1 | Bath Towels     |       25 | pieces  |            10 |
|       2 | Soap Bars       |        8 | pieces  |            10 |
|       3 | Shampoo Bottles |       15 | bottles |             5 |
|       4 | Bed Sheets      |        6 | pieces  |             8 |
|       5 | Water Bottles   |       50 | bottles |            20 |
+---------+-----------------+----------+---------+---------------+
5 rows in set (0.00 sec)
```

Figure 4.7: Inventory table and its records.

5. SQL Queries and Outputs

SQL queries are executed to retrieve, insert, update, delete, and analyse the data stored in the database. A total of 20 queries are used to demonstrate different database operations.

Query 1: Display All Guests

This query displays all the records stored in the Guests table.

```
mysql> SELECT * FROM Guests;
```

OUTPUT:

```
+----------+-------------+------------+-------------------+------------+
| guest_id | full_name   | phone      | email             | city       |
+----------+-------------+------------+-------------------+------------+
|        1 | Aarav Sharma | 9876500001 | aarav@example.com | Hyderabad  |
|        2 | Priya Reddy  | 9876500002 | priya@example.com | Vijayawada |
|        3 | Rahul Kumar  | 9876500003 | rahul@example.com | Warangal   |
|        4 | Sneha Rao    | 9876500004 | sneha@example.com | Hyderabad  |
|        5 | Arjun Patel  | 9876500005 | arjun@example.com | Chennai    |
+----------+-------------+------------+-------------------+------------+
5 rows in set (0.00 sec)
```

Figure 4.8: Output for displaying all guests.

Query 2: Display Available Rooms

This query displays the rooms that are currently available for booking.

```
mysql> SELECT room_number, room_type, price_per_night
    -> FROM Rooms
    -> WHERE room_status='Available';
```

OUTPUT:

```
+-------------+-----------+-----------------+
| room_number | room_type | price_per_night |
+-------------+-----------+-----------------+
| 101         | Single    |         1500.00 |
| 202         | Suite     |         6500.00 |
| 302         | Deluxe    |         4200.00 |
+-------------+-----------+-----------------+
3 rows in set (0.00 sec)
```

Figure 4.9: Output for available rooms.

Query 3: Find Rooms Costing More Than ₹3,000

This query displays rooms whose price per night is greater than ₹3,000.

```
mysql> SELECT room_number, room_type, price_per_night
    -> FROM Rooms
    -> WHERE price_per_night > 3000
    -> ORDER BY price_per_night DESC;
```

OUTPUT:

```
+-------------+-----------+-----------------+
| room_number | room_type | price_per_night |
+-------------+-----------+-----------------+
| 202         | Suite     |         6500.00 |
| 302         | Deluxe    |         4200.00 |
| 201         | Deluxe    |         4000.00 |
+-------------+-----------+-----------------+
3 rows in set (0.00 sec)
```

Figure 4.10: Output for rooms costing more than ₹3,000.

Query 4: Insert a New Guest

This query inserts a new guest record into the Guests table.

```
mysql> INSERT INTO Guests (full_name, phone, email, city)
    -> VALUES ('Isha Nair','9876500006','isha@example.com','Bengaluru');
```

OUTPUT:

```
Query OK, 1 row affected (0.01 sec)
```

Figure 4.11: Output for inserting a new guest.

Query 5: Update Room Status

This query updates the status of room 102 to Available.

```
mysql> UPDATE Rooms
    -> SET room_status='Available'
    -> WHERE room_number='102';
```

OUTPUT:

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Figure 4.12: Output for updating room status.

Query 6: Display Reservations with Guest and Room Details

This query uses joins to display reservation details along with the guest name and room number.

```
mysql> SELECT r.reservation_id, g.full_name, rm.room_number,
    ->        r.check_in, r.check_out, r.booking_status
    -> FROM Reservations r
    -> JOIN Guests g ON r.guest_id=g.guest_id
    -> JOIN Rooms rm ON r.room_id=rm.room_id
    -> ORDER BY r.reservation_id;
```

OUTPUT:

```
+----------------+-------------+-------------+------------+------------+----------------+
| reservation_id | full_name   | room_number | check_in   | check_out  | booking_status |
+----------------+-------------+-------------+------------+------------+----------------+
|              1 | Aarav Sharma | 102         | 2026-10-10 | 2026-10-12 | Checked-In     |
|              2 | Priya Reddy  | 201         | 2026-10-11 | 2026-10-14 | Booked         |
|              3 | Rahul Kumar  | 101         | 2026-10-01 | 2026-10-03 | Checked-Out    |
|              4 | Sneha Rao    | 202         | 2026-10-15 | 2026-10-17 | Booked         |
|              5 | Arjun Patel  | 302         | 2026-09-20 | 2026-09-22 | Checked-Out    |
+----------------+-------------+-------------+------------+------------+----------------+
5 rows in set (0.00 sec)
```

Figure 4.13: Output for reservation details.

Query 7: Display Reservations with Billing Details

This query uses joins to display reservation information along with the corresponding billing details.

```
mysql> SELECT r.reservation_id, g.full_name, b.amount, b.payment_status
    -> FROM Reservations r
    -> JOIN Guests g ON r.guest_id=g.guest_id
    -> LEFT JOIN Bills b ON r.reservation_id=b.reservation_id
    -> ORDER BY r.reservation_id;
```

OUTPUT:

```
+----------------+-------------+----------+----------------+
| reservation_id | full_name   | amount   | payment_status |
+----------------+-------------+----------+----------------+
|              1 | Aarav Sharma |  5000.00 | Paid           |
|              2 | Priya Reddy  | 12000.00 | Pending        |
|              3 | Rahul Kumar  |  3000.00 | Paid           |
|              4 | Sneha Rao    | 13000.00 | Pending        |
|              5 | Arjun Patel  |  8400.00 | Paid           |
+----------------+-------------+----------+----------------+
5 rows in set (0.00 sec)
```

Figure 4.14: Output for reservation and billing details.

Query 8: Count Reservations by Status

This query uses the GROUP BY clause and COUNT function to calculate the number of reservations for each booking status.

```
mysql> SELECT booking_status, COUNT(*) AS total_reservations
    -> FROM Reservations
    -> GROUP BY booking_status
    -> ORDER BY booking_status;
```

OUTPUT:

```
+----------------+--------------------+
| booking_status | total_reservations |
+----------------+--------------------+
| Booked         |                  2 |
| Checked-In     |                  1 |
| Checked-Out    |                  2 |
+----------------+--------------------+
3 rows in set (0.01 sec)
```

Figure 4.15: Output for reservation counts by status.

Query 9: Calculate Total Paid Revenue

This query uses the SUM function to calculate the total amount received from paid bills.

```
mysql> SELECT SUM(amount) AS total_paid_revenue
    -> FROM Bills
    -> WHERE payment_status='Paid';
```

OUTPUT:

```
+--------------------+
| total_paid_revenue |
+--------------------+
|           16400.00 |
+--------------------+
1 row in set (0.00 sec)
```

Figure 4.16: Output for total paid revenue.

Query 10: Calculate Average Room Price

This query uses the AVG function to calculate the average price per night of all rooms.

```
mysql> SELECT ROUND(AVG(price_per_night),2) AS average_room_price
    -> FROM Rooms;
```

OUTPUT:

```
+--------------------+
| average_room_price |
+--------------------+
|            3583.33 |
+--------------------+
1 row in set (0.00 sec)
```

Figure 4.17: Output for average room price.

Query 11: Display Guests Who Made Reservations

This query displays the guests who have made reservations using a join between the Guests and Reservations tables.

```
mysql> SELECT DISTINCT g.guest_id, g.full_name, g.city
    -> FROM Guests g
    -> JOIN Reservations r ON g.guest_id=r.guest_id
    -> ORDER BY g.guest_id;
```

OUTPUT:

```
+----------+--------------+------------+
| guest_id | full_name    | city       |
+----------+--------------+------------+
|        1 | Aarav Sharma | Hyderabad  |
|        2 | Priya Reddy  | Vijayawada |
|        3 | Rahul Kumar  | Warangal   |
|        4 | Sneha Rao    | Hyderabad  |
|        5 | Arjun Patel  | Chennai    |
+----------+--------------+------------+
5 rows in set (0.00 sec)
```

Figure 4.18: Output for guests with reservations.

Query 12: Find Pending Bills

This query displays bills whose payment status is Pending.

```
mysql> SELECT r.reservation_id, g.full_name, b.amount, b.payment_status
    -> FROM Bills b
    -> JOIN Reservations r ON b.reservation_id=r.reservation_id
    -> JOIN Guests g ON r.guest_id=g.guest_id
    -> WHERE b.payment_status='Pending';
```

OUTPUT:

```
+----------------+-------------+----------+----------------+
| reservation_id | full_name   | amount   | payment_status |
+----------------+-------------+----------+----------------+
|              2 | Priya Reddy | 12000.00 | Pending        |
|              4 | Sneha Rao   | 13000.00 | Pending        |
+----------------+-------------+----------+----------------+
2 rows in set (0.00 sec)
```

Figure 4.19: Output for pending bills.

Query 13: Find Inventory Items Requiring Restocking

This query identifies inventory items whose quantity is less than or equal to their reorder level.

```
mysql> SELECT item_name, quantity, reorder_level
    -> FROM Inventory
    -> WHERE quantity <= reorder_level;
```

OUTPUT:

```
+------------+----------+---------------+
| item_name  | quantity | reorder_level |
+------------+----------+---------------+
| Soap Bars  |        8 |            10 |
| Bed Sheets |        6 |             8 |
+------------+----------+---------------+
2 rows in set (0.00 sec)
```

Figure 4.20: Output for inventory items requiring restocking.

Query 14: Display Staff Earning More Than ₹25,000

This query displays staff members whose salary is greater than ₹25,000.

```
mysql> SELECT staff_name, job_role, salary
    -> FROM Staff
    -> WHERE salary > 25000
    -> ORDER BY salary DESC;
```

OUTPUT:

```
+--------------+------------+----------+
| staff_name   | job_role   | salary   |
+--------------+------------+----------+
| Kiran Rao    | Manager    | 45000.00 |
| Anjali Verma | Accountant | 30000.00 |
+--------------+------------+----------+
2 rows in set (0.00 sec)
```

Figure 4.21: Output for staff salary details.

Query 15: Calculate Reservation Nights

This query uses the DATEDIFF function to calculate the number of nights for each reservation.

```
mysql> SELECT reservation_id, DATEDIFF(check_out,check_in) AS nights
    -> FROM Reservations
    -> ORDER BY reservation_id;
```

OUTPUT:

```
+----------------+--------+
| reservation_id | nights |
+----------------+--------+
|              1 |      2 |
|              2 |      3 |
|              3 |      2 |
|              4 |      2 |
|              5 |      2 |
+----------------+--------+
5 rows in set (0.00 sec)
```

Figure 4.22: Output for reservation duration.

Query 16: Find the Most Expensive Room

This query uses a subquery and the MAX function to identify the room with the highest price per night.

```
mysql> SELECT room_number, room_type, price_per_night
    -> FROM Rooms
    -> WHERE price_per_night = (
    ->     SELECT MAX(price_per_night) FROM Rooms
    -> );
```

OUTPUT:

```
+-------------+-----------+-----------------+
| room_number | room_type | price_per_night |
+-------------+-----------+-----------------+
| 202         | Suite     |         6500.00 |
+-------------+-----------+-----------------+
1 row in set (0.00 sec)
```

Figure 4.23: Output for the most expensive room.

Query 17: Update Bill Payment Status

This query updates the payment status of bill number 2 to Paid.

```
mysql> UPDATE Bills
    -> SET payment_status='Paid'
    -> WHERE bill_id=2;
```

OUTPUT:

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Figure 4.24: Output for updating bill payment status.

Query 18: Delete the Sample Guest

This query deletes the sample guest inserted in Query 4 using the guest's email address.

```
mysql> DELETE FROM Guests
    -> WHERE email='isha@example.com';
```

OUTPUT:

```
Query OK, 1 row affected (0.01 sec)
```

Figure 4.25: Output for deleting a guest record.

Query 19: Calculate Inventory Totals by Unit

This query uses the SUM function and GROUP BY clause to calculate the total quantity of inventory items for each unit.

```
mysql> SELECT unit, SUM(quantity) AS total_quantity
    -> FROM Inventory
    -> GROUP BY unit;
```

OUTPUT:

```
+---------+----------------+
| unit    | total_quantity |
+---------+----------------+
| pieces  |             39 |
| bottles |             65 |
+---------+----------------+
2 rows in set (0.00 sec)
```

Figure 4.26: Output for inventory totals by unit.

Query 20: Display Guests with Bills Above ₹5,000

This query uses joins to display guest names, reservation IDs, and bill amounts greater than ₹5,000.

```
mysql> SELECT g.full_name, r.reservation_id, b.amount
    -> FROM Guests g
    -> JOIN Reservations r ON g.guest_id=r.guest_id
    -> JOIN Bills b ON r.reservation_id=b.reservation_id
    -> WHERE b.amount > 5000
    -> ORDER BY b.amount DESC;
```

OUTPUT:

```
+-------------+----------------+----------+
| full_name   | reservation_id | amount   |
+-------------+----------------+----------+
| Sneha Rao   |              4 | 13000.00 |
| Priya Reddy |              2 | 12000.00 |
| Arjun Patel |              5 |  8400.00 |
+-------------+----------------+----------+
3 rows in set (0.00 sec)
```

Figure 4.27: Output for guests with bills above ₹5,000.